Vox Conversation Mode: Product Requirements

Punt Labs

August 2026

Contents

Chapter 1

Product Requirements

1.1 Overview

Vox today speaks: an agent calls `mic:unmute` and text becomes audio. Conversation Mode adds the other half. A human runs `vox call start` (or `/call start` from within a text conversation), talks naturally, and a Claude Code agent session hears them, thinks, and answers out loud — a phone call with an agent, hands and eyes free, while the human keeps working, walking, or looking away from the screen.

This is not a new product. It is a mode of the existing `voxd` daemon: the same process that owns playback and the music program grows the ability to also own capture, turn-taking, and a live call. Nothing about Conversation Mode changes how vox speaks when it is not in a call.

The best version of this feature is measured by one thing: whether a human would rather have this conversation than type it. That bar is not met by transcription accuracy or voice quality alone — both existing TTS providers already clear that bar. It is met by the call *feeling* alive during the several seconds an agent spends reading files and running tools, and by an interruption landing as fast as it would with a person on the other end of a phone.

Primary listening scenario: headphones. The default assumption for Conversation Mode is a human wearing headphones. This is a product decision, not an incidental detail: with headphones, the agent's voice never reaches an open microphone, so the hardest problem in a full-duplex call — acoustic echo between the speaker and the mic — does not exist for the primary scenario. A secondary scenario, a user on a laptop or desktop speaker at ordinary office distance without headphones, is real and in scope, but it is sequenced after the headphone experience is proven, not solved at the same time.

1.1.1 Priorities and phasing

Conversation Mode ships in two phases, not as one undifferentiated feature:

Phase 1 — prove the bar ([p1]). One provider, ElevenLabs, used for both speech recognition and speech synthesis. One listening scenario, headphones. One platform, macOS. The goal of Phase 1 is narrow on purpose: prove that a call with the best available provider, in the easiest audio configuration, is good enough that a person would choose to talk instead of type. Everything that makes a call *feel* alive — turn-taking, barge-in, the agent's think-time presence, sentence-level response streaming — is proven here, against a single provider, before it is asked to also work across providers or platforms.

Phase 2 — broaden it ([p2]). Local providers on macOS and Linux, the no-headphones of-

fice scenario, and any additional cloud provider. Phase 2 is built against the interfaces Phase 1 already defined, not a redesign — see **FR-21**.

Every use case and functional requirement below carries a [P1] or [P2] tag. A [P2] tag is a sequencing decision, not a statement that the requirement is optional or unimportant.

1.1.2 Goals

- [P1] A human can start, hold, and end a natural spoken conversation with one Claude Code agent session, hands-free after starting the call, wearing headphones, with ElevenLabs configured.
- [P1] The experience is excellent on that path — Conversation Mode is designed to its best provider first, not down to the weakest one it will eventually support.
- [P2] A user who wants a fully private conversation — no audio leaving the machine — can have one, on both macOS and Linux, by choosing a local provider configuration.
- [P2] The feature works on both macOS and Linux, and without headphones, though not necessarily with identical provider options or identical audio handling in every configuration.

1.1.3 Non-Goals

- Multi-party calls (more than one human, or more than one agent).
- Calls that span multiple agent sessions or hand off between agents.
- Telephony (PSTN, dial-in numbers, SMS).
- Video or any non-audio modality.
- Provider-side conversational orchestration (e.g. ElevenLabs Agents Platform or OpenAI's own reasoning model standing in for the Claude Code agent). The agent's reasoning is always a Claude Code session; cloud providers supply audio transport only.
- Automatic mid-call failover between providers, or any other runtime resilience behavior. A call uses the provider configuration it started with; if that provider becomes unusable, the call ends with a clear reason, the same way vox's existing TTS surface fails today rather than silently switching providers underneath the user. This also covers a provider that degrades rather than fails outright (slow, glitchy, briefly unresponsive) mid-call: the system does not detect or compensate for degraded-but-still-functioning service — the same scope cut, applied consistently.
- A persisted, reviewable record of a call after it ends. Conversation Mode optimizes for the live conversation; a post-call transcript is a plausible fast-follow, not a launch requirement, and raises its own question (does a transcript get written to disk even in the fully local, no-network configuration?) that is better answered when it is actually being built.
- Changing how vox behaves outside an active call.

1.1.4 Personas

The pair-programmer Runs Claude Code all day, wants to describe a bug or ask a question without stopping to type, especially when their hands are occupied or they are away from the keyboard. Wears headphones most of the working day already — the primary persona for Phase 1.

The privacy-conscious user Wants the option to hold the entire conversation on-device, with no audio leaving the machine, matching the posture vox already offers via local TTS providers. Phase 2.

The Linux user Expects the same feature, accepts that the platform-native convenience some macOS users get is not available, and should never be told the feature simply does not exist on their platform. Phase 2.

1.2 Use Cases

Each use case names the actor, the trigger, the expected flow, and what "done well" looks like. Functional requirements in §1.3 cite the use cases they satisfy.

UC-1 Start a call. [p1] *Actor: any user.* The user is working in an editor or terminal with an active Claude Code session already open, and types `/call start` into that session. The call begins listening within a perceptible instant; no setup dialog, no mode selection, is required if a working configuration already exists. *Done well:* the user does not have to think about which provider is about to answer them, and the agent they are now talking to has the context of what they were just doing, not a blank start — see **FR-4**'s 2026-08-23 revision for exactly what "has the context" means as currently designed (a snapshot taken at call start, not a live link to the session's ongoing state).

UC-2 Ask a question and get a spoken answer. [p1] *Actor: any user.* The user speaks a complete thought, pauses, and the agent processes it and answers audibly. *Done well:* the human never wonders whether the system heard them, and the reply begins as soon as the agent has anything to say — not only once the agent has finished saying all of it.

UC-3 Interrupt the agent mid-answer. [p1] *Actor: any user.* The agent is mid-sentence; the human starts talking over it, either to correct a wrong assumption or to redirect. *Done well:* the agent's voice stops as fast as a human would stop talking when interrupted, and what the human said while interrupting is not lost. On headphones this does not require echo cancellation to work reliably — the mic only ever hears the human, never the agent's own voice.

UC-4 Pause mid-thought without ending the turn. [p1] *Actor: any user.* The human stops speaking briefly — to think, to cough, to react to something in the room — without intending to hand the turn to the agent. *Done well:* the system does not prematurely send a half-formed thought, and does not require the human to learn a special cadence of speech to avoid this.

UC-5 Wait through a long agent turn. [p1] *Actor: any user.* The agent's answer requires reading files, running a search, or several tool calls, taking anywhere from a few seconds to tens of seconds. *Done well:* the call never goes silent in a way that makes the human wonder if it dropped; something — an acknowledgment, a narrated step, an audible cue — signals the agent is still working.

UC-6 Hold a fully private conversation. [p2] *Actor: the privacy-conscious user.* The user has configured only local providers, or has no network connection at all. *Done well:* the call functions end to end with no audio or transcript leaving the machine, on both macOS and Linux, even though voice quality and latency may be noticeably lower than the cloud experience.

UC-7 Run the feature on Linux. [p2] *Actor: the Linux user.* The user has no cloud provider configured and no macOS-only local option available. *Done well:* the feature still works,

using whichever local provider is available on that platform, and any setup step this requires (e.g. a one-time model download) is explained, not silently imposed.

UC-8 Talk without headphones, in an office. [**p2**] *Actor: any user.* The user is on a laptop or desktop, at ordinary conversational distance from the built-in or external mic and speaker — not across a room — without headphones, in a typical office with some ambient noise. *Done well:* the system does not mistake ordinary room noise for speech, does not mistake the agent's own voice coming through the speaker for a human interruption, does not require silence to function, and when it genuinely cannot make out what was said, it asks the human to repeat themselves rather than acting on a guess. This use case is explicitly not about far-field or across-the-room use — that is out of scope entirely, not merely deferred.

UC-9 End the call. [**p1**] *Actor: any user.* The user says goodbye, runs `/call stop` (or `vox call stop`), or simply stops engaging. *Done well:* an explicit hangup ends the call immediately; an inactive call ends itself after a bounded timeout rather than holding the microphone open indefinitely.

UC-10 Start a call for the first time, unconfigured. [**p1**] *Actor: a new user.* No provider has been configured yet and the user runs `/call start`. *Done well:* the user is told what is needed to get a call working (an ElevenLabs key, in Phase 1) rather than the call silently failing to start or starting into a broken state.

UC-11 Start a call the configured provider cannot reach. [**p1**] *Actor: any user.* The user runs `/call start` while their configured provider is unreachable — no network, an expired key, a provider outage. *Done well:* the call does not start, and the user is told why, immediately and in a way they can act on, rather than being dropped into a call that will not work.

1.3 Functional Requirements

Requirements are grouped by capability area. Each cites the use case(s) it exists to satisfy and carries the same [P1]/[P2] phase tag.

1.3.1 Call lifecycle

FR-1 [**p1**] The system shall let a user start a call with a single action (`vox call start` or `/call start`), with no required configuration step beyond what already makes vox speak today. (**UC-1**) When no provider is configured, the system shall tell the user what is needed rather than failing silently. (**UC-10**)

FR-2 [**p1**] The system shall let a user end a call explicitly at any time, and shall end an inactive call automatically after a bounded timeout. (**UC-9**)

FR-3 [**p1**] **Revised.** The system shall communicate every call-state transition (ready to listen, agent finished speaking, call ending) *audibly*, through the same voice channel already used for replies. A visual or text signal is not sufficient on its own: live testing (Spike 5) confirmed that a hands/eyes-free user genuinely does not see on-screen text, so a state signal that depends on being read is not a signal at all for the primary scenario. The original wording ("somewhere the user can check it") is too weak and is superseded by this. (**UC-1**, **UC-5**)

FR-4 [**p1**] A call shall involve exactly one human and one Claude Code agent session for the lifetime of that call, and that session shall be the user's active session — carrying whatever

task context it already had — not a newly started, context-free session. **Revised (2026-08-23).** The mechanism ratified for **FR-4** (Decision 1, §2.2) has changed twice since this requirement was written, and the second change narrows what "the user's active session" can literally mean. The per-turn `claude -p --resume` mechanism (superseded, DES-065) did literally resume the same session object; its replacement (DES-066, a persistent call agent held alive for the call, spiked and confirmed working this session) does not — it starts a fresh process and is handed a *context snapshot* (current task, relevant files, recent conversation gist) as its opening message, deliberately disconnected from the primary session's own live state so a call cannot hang on a primary session that is mid-turn. This still satisfies **FR-4**'s underlying intent (the call carries real task context, not a blank start) but not its literal wording (the *same* session object). **RATIFIED 2026-08-24**, operator: "yes, I sign off on context snapshot not literal session resume." **FR-4** is satisfied by the context-snapshot reading going forward — see §2.2's updated resolution and Decisions required before implementation item 1. (**UC-1**)

1.3.2 Hearing the human

FR-5 [**p1**] The system shall detect when the human has finished a turn, without requiring an explicit "done" signal from the human. (**UC-2**)

FR-6 [**p1**] The system shall not end a turn on an ordinary thinking pause, a cough, or an incidental noise. (**UC-4**, **UC-8**)

FR-7 [**p1**] The system shall not treat steady background room noise as human speech. (**UC-8**)

FR-7a [**p2**] Without headphones, the system shall not treat the agent's own voice audible through an open speaker as human speech. On headphones ([P1]) this requirement is structurally satisfied — the mic never hears the speaker at all. (**UC-8**)

1.3.3 Interruption

FR-8 [**p1**] The system shall let the human interrupt the agent while it is speaking, and shall stop the agent's speech promptly on interruption. **Revised:** "promptly" is two separate, additive latencies (Spike 5), not one number — see **NFR-1**. (**UC-3**)

FR-9 [**p1**] The system shall not discard what the human said while interrupting; that speech shall carry forward as the start of the next turn. (**UC-3**)

FR-10 [**p2**] Without headphones, the system shall distinguish a genuine interruption from the agent's own voice leaking into the microphone, so an interruption is never falsely triggered by the system hearing itself. Not required on headphones ([P1]) for the same structural reason as **FR-7a**. (**UC-8**)

FR-22 [**p1**] **(new)** The system shall distinguish a deliberate interruption from an incidental conversational reaction (a brief acknowledgment, exclamation, or backchannel sound like "mm") by requiring a minimum duration of sustained speech before treating audio as an interruption. This holds on headphones too — unlike **FR-10**, it has nothing to do with echo; a human can react audibly while wearing headphones. Spike 5 found this by accident (an operator's unintentional reaction stopped the agent) and could not resolve it from acoustic-duration statistics alone; a working threshold (approximately 500 ms of sustained speech) was reached only through direct live tuning against human feedback, and that number is one operator's preference in one session, not a validated default. See Chapter 2 for the mechanism and §1.5 for what remains open. (**UC-3**)

1.3.4 The agent's turn

FR-11 [**p1**] The system shall begin speaking the agent's reply as soon as the first complete portion of it is available, rather than waiting for the entire reply to be ready. (**UC-2**, **UC-5**)

FR-12 [**p1**] **Revised.** While the agent is working on a reply, the system shall play a pre-authored, bundled ambience loop (quiet workspace foley — typing, paper shuffle, room tone) on the daemon's existing background audio channel, independent of the foreground speech channel. The original candidate mechanism — `mic:music`'s live Music-generation API, ducked under the call — is superseded: Spike 4 found the Music-generation model produces repetitive, non-representative content for foley-type sound (it is tuned for melody, not realistic ambience) and requires expensive per-refresh runtime generation. The corrected mechanism is described in Chapter 2. (**UC-5**)

FR-13 [**p1**] The microphone shall remain available to the human throughout the agent's turn, so a human who wants to add or correct something is never locked out. (**UC-3**, **UC-5**)

1.3.5 Providers

FR-14 [**p1**] The system shall support ElevenLabs as a Conversation Mode provider, for both speech recognition and speech synthesis, on macOS. (**UC-1**)

FR-14a [**p2**] The system shall additionally support at least one fully local speech provider on each of macOS and Linux. (**UC-6**, **UC-7**)

FR-15 [**p1**] The system shall let a user configure which provider is used for Conversation Mode, consistent with how providers are already configured for vox's existing text-to-speech surface. A call uses the configuration it was started with for its entire duration. (**UC-1**)

FR-16 [**p2**] The system shall not require any cloud account, API key, or network connection to hold a complete call on either supported platform, when configured for a local provider. (**UC-6**, **UC-7**)

FR-17 [**p2**] The system shall clearly account for any setup cost unique to a platform's local provider (for example, a one-time model download on Linux) rather than presenting it as a silent limitation. (**UC-7**)

FR-18 [**p1**] If the configured provider cannot be reached when a call is started, the system shall fail to start the call with a clear reason, rather than starting the call and finding out mid-conversation. (**UC-11**)

FR-21 [**p1**] The interfaces between voxd's call logic and a speech provider (capture, recognition, turn-taking signals, synthesis) shall be defined without assuming ElevenLabs is the only possible implementation behind them, so that Phase 2 providers can be added later without renegotiating the interface boundary. Phase 1 still ships exactly one implementation behind that interface — the abstraction is scoped now so it does not have to be retrofitted, not because Phase 1 needs more than one provider to prove its point.

1.3.6 Feedback and trust

FR-19 [**p1**] The system shall never fabricate or guess at what the human said when audio is ambiguous or capture fails; it shall ask the human to repeat themselves rather than act on a low-confidence guess. (**UC-8**, and generally whenever capture confidence is low)

1.4 Non-Functional Requirements

NFR-1 [p1] (Interruption latency) Revised, validated with real data. Total perceived interruption latency is two additive, separately-measured quantities, not one number:

1. *Detection latency* — sustained speech required before the system treats audio as a deliberate interruption (**FR-22**). Validated live: 500 ms was tuned against direct human feedback and judged "good enough" by the operator. This is a starting default, not a proven-optimal or universal constant.

2. *Termination latency* — from the stop decision to audible silence. Spike 2 measured process-kill time alone (0.3–5.3 ms, real hardware, real interruption) and mistook it for the whole quantity; Spike 5 found that killing the playback process does not guarantee already-buffered audio stops immediately, and this buffer-tail component was never measured. **Open**, not closed — see §1.5.

The original single "under 300 ms" target is superseded: it did not distinguish these two quantities, and the validated detection component alone (500 ms) already exceeds it. (Supports **FR-8**, **FR-22**.)

NFR-2 [p1] (Time to first spoken word) Component legs confirmed; system-level number now measured too, and it falsifies the target. The 580–683 ms figure below was, and remains, correct for what it measured: the audio-facing legs (STT, TTS, detection) with the agent role played by hand by the authoring session, per §2.2's own caveat that this was "provisional for the system as a whole." That system-wide number is no longer provisional — Slice 1b (vox-gs9u.2) measured it for real, through the actual ratified session-attach mechanism (§2.2, Decision 1), on 2026-08-23: a 20-run benchmark (`claude -p`, both a resumed real session and a fresh session, sonnet and haiku, no accumulated context) put time-to-first-word at a **13–25 s median**, not 580–683 ms — one to two orders of magnitude over target, dominated by process-spawn and hook-bootstrap cost external to the agent's own reported turn duration (a fresh-session run reported `duration_api_ms: 2243` internally against a 13.2 s external wall clock). **Per-turn subprocess spawn is confirmed unworkable for this NFR and is off the table.** See the resolution note in §2.2 and §2.9 for the two-track response: a near-term mitigation (a spoken acknowledgment on turn-end, covering the wait, tracked as vox-36xc) and a longer-term redesign investigation (a persistent, non-per-turn call agent, tracked as vox-hobl). The original component-level measurement below is retained as evidence the audio pipeline itself is not the bottleneck — the session-attach mechanism is. (Supports **FR-11**.)

Original component-level measurement, retained for reference: measured end to end against a real agent and real tool-use turns (Spikes 3 and 5, 5 trials total): 580–683 ms, flat regardless of total reply length (the longest reply in Spike 3 had the lowest latency). Spike 1 separately measured the ElevenLabs TTS provider floor in isolation, with no agent in the loop — 145–162 ms across 5 trials — which is a different quantity (provider latency alone) supporting the same conclusion from a different angle: it establishes the headroom the end-to-end number above has to work with, not an additional sample of the same measurement.

NFR-3 [p2] (Platform and provider breadth) Phase 1 targets macOS with ElevenLabs only. Phase 2 extends the same interfaces to Linux, to local providers on both platforms, and to the no-headphones scenario; the platforms and providers are not required to reach that experience through identical mechanisms.

NFR-4 [p2] (Privacy posture) In a fully local configuration, no audio or transcript content shall leave the machine at any point during a call.

NFR-5 [p1] (No regression to existing behavior) Vox's behavior when Conversation Mode is not active shall be unchanged by this feature's existence.

NFR-6 [p1] (new) (Audible-only state signaling) No call-state transition (**FR-3**) may depend on a visual or text channel as its only signal. A generic, unverified audio cue is not sufficient either: Spike 5 found a standalone system chime unreliable for a reason never root-caused, while vox's own TTS voice (`mic:unmute`), already proven all session, worked every time. The call's own voice channel is the only channel this document treats as trustworthy for state signaling until an alternative is separately verified.

1.5 Open Questions

- **Termination latency's buffer-tail component** (**NFR-1**). Process-kill time is measured (0.3–5.3 ms); the time from kill signal to actual audible silence, inclusive of any output buffer already handed to the OS, is not. This likely requires controlling playback through a continuously-owned audio stream rather than spawning and killing a player subprocess per segment — see Chapter 2, §2.5.
- **Is 500 ms the right default for FR-22, and should it be tunable per user or per session?** Reached through one operator's live feedback in one room on one day. Reasonable as a shipped default given how it was derived, but not validated across speakers, rooms, or microphones.
- **The audible ready-to-listen cue is unresolved. NFR-6** establishes that it must be audible and that vox's own TTS voice is the only channel proven reliable; **FR-3**'s "ready to listen" transition specifically was never made to work reliably in Spike 5 using a non-speech chime. Does the listening cue need to be spoken (costing a beat of latency and a few words of vocabulary) or is there a non-speech sound that can be independently verified as reliable before Phase 1 ships?
- **Multi-session audio contention.** During Spike 5, other concurrent Claude Code sessions on the same machine were suspected (incorrectly, on investigation) of bleeding audio into the call's microphone. The suspicion was wrong in that instance, but the underlying question — what happens to a call when another vox client on the same daemon plays audio at the same time — was never actually tested and is not addressed anywhere in this document.
- Is a one-time local-model download (**FR-17**) acceptable as part of vox's existing install story, or does it need its own onboarding step? (Phase 2 question; not blocking Phase 1.)
- **FR-18** (fail closed at call start) and the mid-call-degradation non-goal are a deliberate simplification to keep scope small — is that the right call, or does early user feedback push toward resilience sooner than expected?
- How many ambience loops (**FR-12**) ship in the bundle, of what length, and does the presence mechanism need a fourth `Format` in the existing Program state machine or a simpler static-asset path that bypasses it? Both are architecturally supported (Chapter 2, §2.7); the choice is undecided.

Chapter 2

Technical Design Solution

Chapter 1 states what Conversation Mode must do. This chapter states how, at the level of components and data flow rather than code — synthesized from five live spikes (`docs/conversation-mode-spikes.m` not from advance design. Where a spike left something unresolved, this chapter says so rather than papering over it; the open items here are the same items in §1.5, restated as design gaps instead of questions. This chapter is still not the Z specification — per `docs/WORKFLOW.md`'s stateful-audio gate, the call state machine (**FR-4**'s single-session lifecycle, idle/listening/waiting/speaking) gets its own formal model before implementation. This chapter is the design that model will formalize.

2.1 Component overview

A call is one continuously-running loop with five components, each independently proven by a spike:

Component	Proven by	Real measurement
Turn detection (capture → end-of-turn)	Spikes 2, 5	see §2.4
Speech recognition (audio → text)	Spike 1	2.2–2.5 s to first partial
Agent turn (text → reply)	Spike 3, 5	N/A — Claude Code itself
Sentence-streamed synthesis (reply → audio)	Spike 3	580–683 ms first word
Barge-in (audio → stop signal)	Spikes 2, 5 (partial)	see §2.5

None of these components block another: turn detection and barge-in detection are both continuous, audio-only processes that run for the entire life of a call, independent of which of the other three components is currently active. **Correction:** an earlier draft of this document claimed this meant Spike 5 had "no orchestration layer to get wrong." That overstates it — Spike 5's own findings include a process-coordination bug (finding (a)), an audio-routing bug (finding (b)), and a barge-in test harness that produced meaningless data because it never closed the loop between detection and playback (finding (e)), all genuinely orchestration-level failures between otherwise-correct components. The corrected claim is narrower: the five components' *internal* correctness was independently established by different spikes, which is why fixing each one individually (rather than debugging an entangled whole) was possible — not that wiring them together was uneventful.

Design requirement added after Chapter 3's test-plan review: these components being independent and continuous means they can call into the shared call state machine (§2.2 onward) concurrently — turn detection, barge-in detection, and sentence-streamed synthesis are three

separate processes that may all observe or attempt to change call state at overlapping moments. If the state machine is implemented with `voxd/programs/program.py`'s `Program` shape (recommended — see the design-mission note in §2.5), that shape is only safe under `Program`'s own documented assumption of a single serializing writer; `Program` itself has no internal lock. **The call state machine must be reachable only through one serialized dispatch point** (a queue, an actor, or an explicit lock at the call boundary) — this is a requirement on the orchestration layer wrapping the state machine, not something the pure state class can guarantee by construction. Where exactly that serialization point lives is part of what §2.2's Z specification must settle, not an implementation detail to discover afterward.

2.2 The unresolved core: session attachment

Before any other mechanism in this chapter, one gap has to be named plainly, because everything else described here is downstream of it and an earlier draft under-weighted it into a single bullet in Known Unknowns: **how a transcribed turn reaches, and how a streamed reply is pulled from, the user's already-running Claude Code session is undesigned.** FR-4 requires the call to use the user's active session — carrying its existing task context — not a fresh one. Every spike that stood in for "the agent" actually used *this authoring session itself*, driven by hand: Spike 3's real agent turns were "seeded by a genuinely multi-second real turn in this session," and Spike 5's integration loop ran inside the live authoring conversation with the operator. No spike attempted programmatic injection of transcribed audio into an independent, already-running session, nor programmatic extraction of that session's streamed reply. Turn detection, barge-in, and sentence-streamed synthesis are all real, all measured, and all irrelevant if there is no mechanism to actually connect them to a live agent session that isn't the one authoring this document. **This needs at least a sketch of the injection/extraction API shape — is it a hook into the Claude Code session surface, a queue the session polls, does it require the session to run inside `voxd`'s process space or communicate over IPC — before an implementation mission dispatches,** even if that sketch is only an ADR-style list of rejected and chosen alternatives, not a full design.

One direct consequence for the rest of this chapter: **every latency number reported in NFR-1 and NFR-2 was measured with the agent role played by this authoring session, not through whatever session-attach mechanism is eventually built.** Those numbers establish that the audio-facing legs of the pipeline (STT, TTS, detection) are fast enough — they do not yet establish that the full system will be, once a real attach mechanism's own latency is added. Treat **NFR-1**/**NFR-2** as validated for the components measured, provisional for the system as a whole.

Resolution (2026-08-23). The session-attach shape was designed (the ADR below, Decision 1: a headless `claude -p --resume` subprocess per human turn), implemented, and shipped in Slice 1b (vox-gs9u.2). The *provisional* qualifier above is now *resolved, negatively*: real end-to-end testing (a live operator call plus a 20-run repeatable benchmark, both documented in **NFR-2**) found the per-turn subprocess design costs 13–25 s median per turn, one to two orders of magnitude over the component-level number this section warned might not hold. The dominant cost is not the model call — it is process spawn plus Claude Code's own SessionStart hook cascade (9–28 hooks observed per invocation depending on resume-vs-fresh, entirely orthogonal to this feature's own audio pipeline). **Spawning a fresh agent process per conversational turn is confirmed unworkable and is off the table going forward.** Two tracks follow from this, both tracked outside this document in beads rather than re-litigated here: a near-term mitigation to make the current per-turn design tolerable (vox-36xc: a spoken acknowledgment on turn-end, background audio during the wait, and a stripped-down harness invocation to shave

what latency is honestly available) and a longer-term architecture investigation (vox-hobl: a persistent call agent kept alive for the whole call instead of respawned per turn, running read-only against the codebase/project, with any codebase-write consequences of the call deferred to a summary the primary session applies after the call ends). The second track is a genuine fork from Decision 1 as ratified and needs its own design pass and operator ratification before implementation, per this project's own workflow discipline — it is named here, not designed here.

Further resolution (2026-08-23, same day). vox-hobl's architecture investigation ran three narrow spikes, each aimed at killing the approach quickly if it could not work rather than discovering that a day into implementation — `DESIGN.md` DES-066 has the full evidence trail; summarized here:

1. **Persistence works.** `pi --mode rpc --no-session` held one process alive across two sequential turns, replying correctly to both, with the second turn's own usage report showing it reused the first turn's warm context (`cacheRead`) rather than re-establishing it from nothing. The first attempt reproduced a real hang (a bundled `biff-bridge` extension interfering with the RPC stdout stream); `--no-extensions` resolved it outright — root-caused, not routed around.
2. **Real tool use holds the latency win.** Re-run with actual file-read questions against this repo instead of canned replies: a cold turn including a real tool call reached first spoken text at 6.8 s and completed at 7.0 s; a warm follow-up turn completed 4.3 s later. Both an order of magnitude under the 13–25 s per-turn-spawn median DES-065 measured, and that number did zero tool use.
3. **Read-only enforcement resolved at the proportionate level.** The operator's own scoping narrowed this from "rock solid" to "the call agent must not collide with repo files a human might be editing concurrently through the primary session" — it does not need to guard against anything touching the primary session's own state, since the two are already disconnected by design. `pi`'s native `--tools read,grep,find,ls` CLI allowlist (confirmed against `pi`'s own documentation, not guessed) satisfies this: verified live by prompting the agent to write a probe file with only that allowlist set — no write tool was invoked, and the file never appeared on disk.

The mechanism is confirmed workable. **Further resolved 2026-08-24 (DES-067):** the two smaller pieces this paragraph originally left open are now decided. The context snapshot is authored by the *primary* session itself at `/vox:call start <topic>` — the session already holds the task context; handing it off does not need a retrieval mechanism. Quarry and context7 lookups are folded into the snapshot the same way, run by the primary session before the call agent spawns (an initial MCP-in-`pi` approach was spiked and explicitly rejected by the operator — "do not bloat pi"). The post-call summary is a structured document (modeled on `../prfaq/`'s meeting-summary format: Decisions Made, Action Items, Context Gathered, Beads Touched, Notes), authored by the call agent itself and read directly by the still-present primary session when the call ends — no hook needed. See DES-067 for the full decision record and rejected alternatives.

RATIFIED 2026-08-24: the operator has signed off on **FR-4**'s revised wording ("yes, I sign off on context snapshot not literal session resume"). Nothing remains before the implementation mission (`vox-hobl.2`) dispatches except the process-supervision choice already recommended (a direct `subprocess.Popen` held by `voxd` itself, over `tmux`/`pi-tools`' `keep` extension, to avoid a new runtime dependency for a headless daemon use case `keep` was not built for) — see DES-066 and `docs/conversation-mode-call-agent.tex`'s Z model for the process lifecycle those pieces attach to.

2.3 Decisions required before implementation

Three forks in this chapter are explicitly reserved for the operator, per this project's own workflow rules (design review must surface substantive issues *before* an implementation mission dispatches, not let them surface as defects during it). Collected here so none of the three can be missed by being one bullet among many elsewhere in this chapter:

1. **Session attachment** (§2.2). **RATIFIED 2026-08-24 — CLOSED.** Ratified 2026-08-22 as the session-attach ADR's Option D (a headless `claude -p --resume` subprocess per human turn); implemented, shipped in Slice 1b (vox-gs9u.2), and confirmed unworkable by real measurement (13–25 s median per turn) on 2026-08-23 — see §2.2's resolution note. The follow-on investigation (vox-hobl: one persistent `pi --mode rpc` call agent held alive for the whole call, not respawned per turn) was spiked and confirmed working across the three dimensions that could have killed it (process persistence, real latency under actual tool use, read-only enforcement) — see the further resolution note above and `DESIGN.md` DES-066. The one remaining operator decision this mechanism raised — that it starts the call agent as a fresh process with a context *snapshot*, not a literal resume of the primary session, satisfying **FR-4**'s intent but not its original literal wording — is now ratified: "yes, I sign off on context snapshot not literal session resume" (operator, 2026-08-24). Nothing further is reserved for the operator on this fork; `vox-hobl.2` (the implementation mission) is unblocked.
2. **Continuous-stream playback ownership** (§2.5): built into production `PlaybackQueue` directly, or as a new sibling component that reuses its diagnostics without inheriting its subprocess-per-item architecture. The first risks **NFR-5** by modifying code every existing vox feature depends on; the second risks the "two pipelines" duplication this chapter argues against elsewhere. No recommendation offered; the trade-off is genuinely close.
3. **Presence-signal mechanism** (§2.7): a fourth Program `Format`, or a simpler static-asset path outside the `Program` state machine. Unlike the other two, this chapter's own analysis leans one way and should say so rather than leave the trade-off unresolved: **recommend the static-asset path.** The `Format` path's only argument is consistency with the existing state machine, but that machine's retry and generation semantics exist to handle a Part that can fail to generate — meaningless for a bundled asset that already exists on disk and cannot fail to generate. Consistency with machinery that would sit entirely unexercised is not a real argument for carrying it. This is a recommendation for the operator to confirm or overrule, not a decision already made.

2.4 Turn detection

Model: accumulated audible-run duration, not a consecutive-chunk streak. The first implementation in Spike 5 tracked consecutive strong chunks and failed twice: too permissive (2 consecutive chunks, 60ms) let transients (coughs, clicks) end a turn falsely; too strict (14 consecutive chunks, 420ms, an overcorrection) missed genuine continuous speech, because real speech has brief amplitude dips between syllables that reset a strict consecutive counter. The corrected model accumulates audible duration across a *run*, resetting only on a genuine silence gap ($\geq$200 ms) — brief within-word dips do not reset it. This mirrors Unramble's `LiveTurnPauseDetector` design (researched earlier in this project) more closely than the first port did; the first port's failure was simplifying that design past the point where it still worked, not a flaw in the underlying approach.

Calibration must precede tuning, not substitute for it. Every threshold in this system (audible ceiling, strong ceiling, sustain duration) is relative to an adaptively-tracked noise

floor, not an absolute constant — but the *multipliers* against that floor were initially guessed from a prior spike's numbers, in a different room, and failed silently (returning STT tags like `[background noise]` rather than an error) until real RMS data was captured (`calibrate.py`: median 0.035, peak 0.377, real silent gaps mid-utterance) and the model was fixed against evidence. A production implementation needs an explicit calibration step — even a few seconds of "say something" at call start — rather than shipping fixed multipliers tuned once in one room.

Scoping gap: this model is tuned against continuous human speech's amplitude profile, and it is not established whether that profile is reliably distinguishable, by duration and gap statistics alone, from bursty non-speech noise — a dog barking intermittently, a knock, irregular nearby typing — which by construction can produce the same "brief gaps within a longer run" pattern the model was built to tolerate. Headphones (P1) remove speaker echo but not ambient room noise reaching an open mic, so this is not automatically a P2-only (no-headphones) concern the way **FR-7a**/**FR-10** are; it is unclear from this design whether P1 also assumes a quiet room, and that assumption should be stated explicitly rather than left implicit.

2.5 Barge-in and its two latencies

Total perceived interruption latency is **detection latency + termination latency**, and they need entirely different engineering:

Detection latency uses the same accumulated-run model as turn detection (§2.4), with a longer sustain requirement (**FR-22**): real interruption behavior sustains for a second or more; backchannel reactions ("mm," a short exclamation) run a few hundred milliseconds. Spike 5's attempts to derive a clean separating threshold from acoustic-duration statistics alone failed for three different, each-diagnosable reasons (methodology flaws in the test, not inconsistency in human behavior — see `conversation-mode-spikes.md` Spike 5(e) for the detail); direct human-in-the-loop tuning (pick a value, ask "too slow or too fast," adjust) reached a working number (500 ms) where statistical derivation did not. **Recommendation: ship this as a live-tunable parameter with a 500 ms default**, not a hardcoded constant — the derivation method that worked once (ask the user) is also the mechanism that should exist at runtime if the default turns out wrong for a given voice or room.

Termination latency is not process-kill latency. Spike 5's prototype stopped playback by sending `SIGTERM` to an `afplay` subprocess per synthesized sentence; Spike 2 measured that kill taking 0.3–5.3 ms and treated the interruption problem as solved. It measured the wrong thing: killing the *process* does not guarantee audio already handed to the OS output buffer stops *sounding*, and this buffer-tail latency was never isolated from the kill-signal latency in any spike. **Design implication: production playback should not shell out to a per-segment player subprocess at all.** It should own a single, continuously-running output audio stream that can be silenced by clearing its buffer directly, not by killing and restarting a process. This also removes the per-sentence subprocess-spawn overhead the spike prototypes paid on every segment.

Only the direction is settled here, not the concurrency discipline around it. "Clear the buffer directly" does not by itself answer: what happens when a barge-in stop-signal arrives at the same moment a segment finishes naturally on its own (a race between turn-completion and interruption)? What happens if a second barge-in fires while the first stop's buffer-clear is still in flight — a stop-during-a-stop? From what thread is the buffer cleared, under what lock, relative to whatever thread is concurrently writing newly synthesized audio into that same stream? This document does not need to answer these at the code level, but per the deferred Z specification noted at this chapter's start, the single-stream design's ordering guarantees are

a required *input* to that formal model, not a detail the model can discover while being written.

Correction: an earlier draft of this document described this as "comparable to `voxd`'s existing `PlaybackQueue`." On inspection, `PlaybackQueue.play_audio()` (`voxd/playback.py:329--466`) spawns one subprocess per file via `asyncio.create_subprocess_exec`, awaits completion or a timeout-based kill, and exposes no mid-playback interrupt method at all — it is architecturally the same subprocess-per-segment shape this section is diagnosing as insufficient, not an existing approximation of the target. `PlaybackQueue` is the right *place* to add a continuous-stream capability, so this feature inherits its existing failure diagnostics instead of duplicating them, but that capability does not exist in production code today and would be new work.

Undecided, and belongs to the operator, not this document: whether that new capability is built *into* `PlaybackQueue` directly, or as a new sibling component that reuses its diagnostics without inheriting its subprocess-per-item architecture. The first risks **NFR-5** (vox's behavior outside a call must be unchanged) by modifying code every existing vox feature — chimes, TTS replies, music — depends on. The second avoids that regression risk but risks the "two pipelines instead of one" duplication this document argues against in the synthesis section below. This is exactly the kind of fork this project's own workflow reserves for an explicit operator decision before an implementation mission dispatches, and it has not been made.

2.6 Sentence-streamed synthesis: one pipeline, not two

Confirmed by Spike 3, built and measured, not merely proposed: vox's existing batch TTS path (`core.py:split_text` → `providers/chunked.py:chunked_synthesize` → `stitch_audio`) and Conversation Mode's incremental path are the same segmentation problem with one difference — whether the full input is known upfront or arrives over time. The design that survived contact with real code: **one segmentation core, built on `split_text`'s actual sentence-boundary regex, with two flush policies on top** — `GREEDY` (accumulate to `max_chars`, matching today's batch behavior byte-for-byte) for the existing TTS surface, and `EAGER` (flush every complete sentence immediately) for Conversation Mode. The naive port failed once, cleanly diagnosed: incremental code has to answer "did the sentence end, or did the input just end?" as a real question, which `split_text` never has to ask because it always sees the whole string. That distinction is exactly what the flush-policy split resolves, without needing two segmenters.

Recommendation: build the real implementation directly on this shape, not on a re-derivation of it — the unification was validated in working code (`.tmp/spikes/conversation-mode/spike3-st` not just argued for.

Gap: the async/sync boundary, which Spike 3 itself named as the dominant unsolved problem, has no design here. `voxd`'s handlers are `async def` throughout, but `chunked_synthesize` and the per-provider synthesis calls it drives are synchronous, blocking calls. Spike 3's own write-up states plainly that bridging this boundary, not the segmenter, is where the real design effort has to go for a production implementation — and that bridging design does not exist yet, here or anywhere else in this document. See §2.9.

2.7 Presence signal: a quips-shaped pool, not a single asset and not `Program`

Spike 4 falsified the original candidate (`mic:music`'s Music-generation API, ducked under the call) on content grounds — repetitive, not representative of real workspace foley, because a music-generation model is tuned for melody and rhythm, not realistic ambient sound — before

the level problem (needed to be turned up loud enough that other audio became uncomfortably loud) was even reached. Two further rounds of design discussion, after the spike, corrected the mechanism twice more; this section states where that landed, not the intermediate steps.

The design settled on a fourth existing vox pattern this document had not previously considered: `quips.py`. `quips.py` holds fixed-size pools of plain text (`STOP_PHRASES`, `IDLE_PHRASES`, and others — seven to ten entries each), picked via `random.choice()` at the moment a hook fires and passed straight into a live synthesis call (`hooks.py:254`). What makes repetition cheap is `cache.py`: audio is content-addressed by `(text, voice, provider)`, so the first time any phrase plays it is a real API call, and every later occurrence of the same phrase is a cache hit. This is a fourth pattern already live in vox, distinct from bundled binary assets (chimes) and from live dynamic generation with retry/pool semantics (`Program`/`Format`, used for Music), and it turned out to be the correct fit here — not either of the two this document previously weighed against each other:

- **A single bundled loop, replayed every call, was rejected on product grounds, not technical ones.** Chimes are fine as static, endlessly-repeated assets because they are short and purely functional — nobody notices a chime repeating. A multi-second ambient loop replayed identically across every call is exactly the kind of content a person does notice, and gets tired of.
- `Program`/`Format` **reuse was rejected for the reason given in an earlier draft of this section** (retry and generation-failure semantics are meaningless for content that is authored once and cached, not generated live per call) **and that reasoning still holds** — but it is now moot rather than a live trade-off, because the quips pattern already solves the actual problem `Program`'s pool/rotate/anti-repeat machinery exists for (avoiding repetition fatigue) at a fraction of the complexity, since nothing here needs to be generated live.

The settled design:

1. **Author a fixed pool of Sound Effects prompts** — discussed as approximately five distinct scenarios — the same shape as a `quips.py` phrase pool, but each entry is a Sound Effects prompt (confirmed purpose-built for foley/ambient sound, with native seamless looping, via ElevenLabs' Sound Effects API) instead of a spoken phrase. **Correction, carried from an earlier draft:** a prior version of this document claimed Sound Effects costs "roughly 1/50th the per-second cost" of Music; that compared a per-second Sound-Effects rate against a per-*track* Music price without converting units, and was wrong — on a like-for-like basis Sound Effects is more expensive per second, not cheaper. The real economic argument is that each pool entry is synthesized at most once per install (see item 2), not the per-second provider rate.
2. **Synthesize each pool entry once, cache via the existing** `cache.py` **infrastructure**, exactly as quip phrases already are — no bundled binary assets shipped in the package, no per-call runtime generation. First use of each scenario on a given machine pays one real synthesis call; every use after that, of any scenario, is a cache hit. Gain-normalization (a live listening test found a generated clip needed +6 dB to be audible at a comfortable level) is a one-time authoring-time step per pool entry, the same as any other quality check on a quip's phrasing.
3. **Select with the same anti-repetition discipline** `quips.py` **already uses** — random choice across the pool, or a simple last-played exclusion, rather than always the same entry or a fixed sequence. This is what actually solves the repetition problem chimes don't have and `Program`'s pool/rotate exists for, without needing `Program`'s generation-failure machinery to get it.

4. **Runs on the existing background channel, independent of speech**, the same channel `Program`-based Music already uses. Confirmed live and by code (`speech_handlers.py` has zero references to `Program`, pause, or duck): `voxd` already runs two independent audio channels, foreground speech/chime (`playback.py`) and background Program (`music_player`). Whether this pool should play through that same background channel via a lightweight sibling to `music_player`, or through a new, smaller mechanism of its own, is not decided by this document; either way, no ducking exists between foreground speech and background audio today, and whether the presence loop needs to duck under speech or can coexist at authored levels is untested.

Out of scope for this design, raised in discussion but not committed as a requirement: a user supplying their own ambience (either an existing audio file, which fits this same cached/static shape with no generation step, or a custom prompt generated on demand, which would need `Program`/`Format`'s actual retry-and-generate machinery, since on-demand generation genuinely can fail transiently the way a fixed authored pool never does). Chapter 1 has no requirement for user-customizable presence audio; if that becomes real scope, the correct design is a hybrid — quips-style pool for the shipped defaults, `Program`/`Format` reuse specifically for user-triggered custom generation — not a reason to revisit the choice made here for the default case.

2.8 Audio-first state signaling

NFR-6 is a design constraint, not just a requirement: **every call-state cue routes through vox's existing TTS voice channel (`mic:unmute`), not a new subsystem.** A standalone system chime (`afplay /System/Library/Sounds/Tink.aiff`) was verified working in isolation but was not reliably heard when invoked from inside the listener process, for a reason not root-caused in this session — plausibly output-device routing differing between a bare shell invocation and one nested inside a Python subprocess, but not confirmed. Switching to the same TTS channel already proven reliable all session worked immediately and consistently. **Do not build a second, unverified audio-cue mechanism when a trusted one already exists** — if a lighter-weight non-speech cue is wanted later for latency reasons, it needs its own dedicated reliability verification before anything depends on it, not an assumption carried over from working once, standalone, outside the real call path. The chime failure's root cause should be understood, not just worked around, before it is attempted again in any form — it is a plausible symptom of an audio-routing issue that could resurface in the presence loop's background channel or any future non-speech cue, not something specific to `/System/Library/Sounds/Tink.aiff` alone.

2.9 Known unknowns

Carried forward from §1.5 as concrete design gaps, not open-ended risk. The first two were missing from this section entirely until an independent design review caught them; they are the largest gaps in this chapter, not minor additions.

- **How a transcribed turn reaches, and how a streamed reply is pulled from, the user's already-running Claude Code session — designed, implemented, measured, and found too slow as designed. Resolved as of 2026-08-23**, superseding the rest of this bullet as written during design: the session-attach mechanism was designed (session-attach ADR, Decision 1: a headless `claude -p --resume` subprocess per human turn), implemented in Slice 1b (vox-gs9u.2), and measured for real. The measurement is the new unknown this bullet leaves open: per-turn subprocess spawn costs 13–25 s median (20-run benchmark, 2026-08-23), dominated by process startup and Claude Code's SessionStart hook cascade, not model latency. Per-turn spawn is confirmed off the table; see

§2.2's resolution note and **NFR-2** for the full data and the two-track response (vox-36xc near-term, vox-hobl for the persistent-agent redesign). **Further resolved 2026-08-23, same day:** vox-hobl's persistent-agent replacement has itself now been spiked and confirmed working (persistence, real tool-use latency, read-only enforcement — see §2.2's further-resolution paragraph and `DESIGN.md` DES-066). **Further resolved 2026-08-24 (DES-067):** the context-snapshot content and the post-call summary handoff, both named as still undesigned as recently as the previous sentence, are now decided — see §2.2's DES-067 pointer above. **RATIFIED 2026-08-24:** the operator has signed off on **FR-4**'s revised wording. This bullet, opened at the start of this chapter's design work, is now fully closed — design, spikes, and ratification all done; `vox-hobl.2` is the implementation mission.

- **Provider-agnostic interfaces (FR-21, P1) have no design content anywhere in this chapter.** Every spike hardcoded ElevenLabs directly — Spike 1 reused `providers/elevenlabs.py` wholesale; Spike 4 bypassed `mic:music` to call the ElevenLabs SDK directly rather than go through any abstraction. **FR-21** requires the capture/recognition/turn-taking/synthesis interface boundary to be defined without assuming ElevenLabs is the only implementation, and nothing in this chapter sketches what that boundary looks like. Silently treating this as solved would be wrong; it has not been attempted.
- **The async/sync boundary** between `voxd`'s `async def` handlers and the synchronous `chunked_synthesize`/provider synthesis calls they currently drive — named by Spike 3 itself as the dominant unsolved engineering problem, ahead of the segmenter — has no design here.
- **Termination's buffer-tail latency** is unmeasured. The architectural fix (§2.5, a continuously-owned output stream instead of per-segment subprocess playback) is proposed but not built or measured.
- **Whether the new continuous-stream playback capability modifies production `PlaybackQueue` directly or is built as a separate component** is an explicit operator decision, not yet made (§2.5).
- **The 500 ms interruption-detection default** is one operator's live-tuned preference, not validated across voices, rooms, or microphones. Ship it as a runtime-tunable parameter, not a constant.
- **Multi-session audio contention** — what a call does when another vox client on the same daemon plays audio concurrently — was never tested. Spike 5 suspected it incorrectly in one instance; the underlying scenario is untested, not disproven.
- **Presence-signal ducking against speech** is untested; the two channels are confirmed independent today, and whether that is sufficient or needs explicit ducking is open.
- **Ambience-Format vs. static-asset path** for §2.7 item 4 is an implementation choice not yet made.
- **FR-19 (never fabricate on ambiguous audio)** has no design coverage here, despite two directly relevant spike findings: Spike 1's ElevenLabs partial transcripts self-correcting mid-stream, and Spike 5's turn detector producing false triggers before it was fixed. How the production system surfaces low-confidence capture to the human is undesigned.
- **Mid-call self-correction.** Spike 3 found that a live agent composing a reply sentence-by-sentence will, at least occasionally, speak a first sentence that a later sentence's reasoning contradicts or walks back — and flagged this explicitly as belonging in the design mission's

open questions. Whether the agent should audibly self-correct mid-call the way a human would is not addressed anywhere in this document.

- **Which detector governs a human speaking during the agent's silent think-time (the waiting state, before any reply audio starts)** is not stated. Barge-in detection is described as active "while it is speaking"; turn detection covers human speech generally; neither section says which one applies, if either, between a transcribed turn being sent and the reply's first audio. This is exactly the kind of state-machine edge case the deferred Z specification needs to resolve, and it should be named before that modeling starts, not discovered during it.
- **A stuck or free-running detector has no described recovery.** Both turn detection and barge-in detection are described as continuous processes running the whole life of a call, but nothing addresses a detector thread that hangs (the call goes silently deaf; **UC-9**'s bounded timeout is a call-level backstop, not a component-health check) or one that free-runs and never accumulates silence (the human's turn never ends). Per **NFR-6**, the TTS voice is the only trusted signaling channel — but a stuck detector by definition produces no signal to send through it, so the human would have no way to know this is happening mid-call.
- **A synthesis call that never returns mid-turn** is unaddressed. **FR-18** covers a provider unreachable *before* a call starts; nothing covers a provider that accepts the call and then hangs on a synthesis request partway through. Likely a one-sentence fix (a timeout on the synthesis call, ending the turn the same way **FR-18** ends the call) rather than new design, but it is not stated anywhere as written.
- **No debugging story for a live call.** A user-facing transcript is an explicit Non-Goal, correctly, for launch scope — but an internal, developer-facing record of state transitions, detector decisions, and per-segment latencies is a different thing and is not mentioned. Given Phase 1's entire purpose is validating a subjective bar ("would a human rather have this conversation than type it"), there is currently no way to determine *why* a given call felt bad — a false barge-in trigger, a slow synthesis segment, a missed turn end — after the fact.
- **Whether anything in this design is actually macOS-specific is not established.** P1 is scoped to macOS, framed as a complexity reduction alongside headphones-only and single-provider. Headphones genuinely eliminate an entire class of problem (echo cancellation); single-provider genuinely defers multi-provider complexity. But nothing in this chapter identifies what part of the design — if any — is actually coupled to macOS versus incidentally macOS because that is where the spikes happened to run. If the design is in fact platform-portable, "macOS-only for P1" is not reducing complexity, it is leaving Linux untested, which is a different (and smaller) claim than the phasing rationale in Chapter 1 makes.

Chapter 3

Test Plan

Chapter 2 designs the system; this chapter designs how it is proven correct, including what has to be *built* to make that possible — this project has no existing test tier for continuous audio-hardware behavior, and two of the new components have no fake/test-double equivalent of the ones already established elsewhere in vox. Grounded in real, present conventions: the four-tier testing pyramid and Humble Object pattern already documented for this project (`TESTING.md`, `punt-kit/standards/python.md`), and the structural, non-inheritance fake style already in use (`tests/_program_fakes.py`'s `FakeProgramGateway`, which records every call and lets a test control results without a live daemon).

3.1 Testing tiers

Tier	What it tests	Status for this feature	Runs in CI
1. Unit	Pure logic: turn/barge-in detectors, the call state machine, segmenter flush policies, presence-pool selection	**Needs new fakes** (§3.2)	Yes
2. Integration	Cross-component wiring: provider protocol conformance, the commands-layer surface, the Z spec's transitions exercised end-to-end in memory	**Needs a fake session-attach double** (§3.2)	Yes
3. Subprocess/E2E	Wire protocol, real provider calls, real (non-human) process lifecycle	**Exists in prototype form only** — every spike script (`.tmp/spikes/conversation-mode/*`) is this tier, unformalized	Optional
4. SDK	End-to-end with a real Claude Code session, costs money	**New for this feature** — prior vox SDK-tier tests never attached to an independent live session; this is Slice 1b's core claim and has no existing harness	No
5. **Live audio, human-judged (new tier)**	Does it *sound* right — barge-in feel, presence-signal naturalness, overall call quality	**Does not exist as a tier today**; this project's audio-demo gate (`docs/WORKFLOW.md`) is the closest precedent but has never been formalized into a repeatable checklist for a *continuous, full-duplex* feature	No, by construction

Tier 5 must be built, not assumed. The project's existing audio-demo discipline (`docs/WORKFLOW.md`) asks a human to confirm audio sounds right, but every prior use of it was for discrete, single-shot playback (a chime, a spoken reply) with an obvious pass/fail. Conversation Mode's audio-demo gate covers a *continuous, two-way* experience — barge-in timing, presence-signal fatigue, whether a call feels alive during a long wait — which needs its own checklist, not a reuse of the existing one by name only. This chapter treats writing that checklist as a required Tier 5 deliverable, owed by Slice 5/Slice 6 (where failure modes and call endings are defined) rather than left implicit.

Tier 3 exists only as five throwaway spike scripts. Each of Spikes 1, 2, 3, and 5 built a real, working subprocess-level test harness against real hardware and real ElevenLabs calls

(`.tmp/spikes/conversation-mode/*`), explicitly marked throwaway per `docs/conversation-mode-spikes.m`. **Recommendation: promote the patterns these scripts already validated** (measuring sentence-1-ready-to-first-audio-byte, measuring detection-to-kill latency, driving a real turn through a real session) into permanent, optional-in-CI Tier 3 tests, rather than re-deriving the same harnesses from scratch during implementation. The scripts are throwaway as *code*; the measurement patterns they proved out are not.

3.2 Design requirements for testability

Each requirement below exists because a component in Chapter 2 currently has no seam for a test double, and the fix is the same one vox's `FakeProgramGateway` and its five-provider `TTSProvider` already demonstrate: a structural (Protocol-shaped, non-inheritance) interface, injected, not owned, by the component under test.

1. **Detectors take PCM chunks as plain method arguments; they never own an audio device.** The turn detector and barge-in detector (§2.4, §2.5) must be constructible and fully testable with zero real microphone access — feed a chunk, assert the signal. This is not a new invention: it is what every spike script already did by hand (`calibrate.py` fed captured PCM into pure functions). The requirement is to keep that shape in production code, not let the detector's constructor reach for `sounddevice` directly the way the spike prototypes did for expedience.

2. **Chunk duration/timestamp is passed as data, never read from the wall clock. Correction, from peer review:** the accumulated-run model (§2.4) depends on real-time durations — a $\geq$200 ms silence gap resets a run, $\sim$500 ms sustained speech triggers an interruption. If the detector reads `time.monotonic()` (or equivalent) internally rather than taking elapsed time as a parameter, every test built against item 1 becomes wall-clock-dependent and flaky — the exact failure mode item 1 already polices for the microphone, missed for the clock. The detector's public interface must accept duration/timestamp explicitly.

3. **An `STTProvider` protocol, following the same *pattern* as `TTSProvider` — not the same shape. Correction, from peer review:** an earlier draft of this chapter said "conforming to the same shape as the existing `TTSProvider`." Read literally that is wrong: `TTSProvider`'s members (`name`, `default_voice`, `supports_expressive_tags`, voice catalog/resolution methods) are half voice-catalog logic with no STT analogue, and `STTProvider` needs partial/streaming transcript delivery (Spike 1's self-correcting mid-stream partials, Chapter 2) and a confidence signal (**FR-19**: ask the human to repeat rather than act on a low-confidence guess) that `TTSProvider` has no counterpart for. The pattern that transfers is: `@runtime_checkable Protocol`, a `check_health() -> list[HealthCheck]` method, one conforming implementation per vendor, structural (not inheritance-based) conformance — not the member list. A fake STT provider built against this returns a scripted transcript *and* a scripted confidence value without a network call, the same way any `TTSProvider` test substitutes a fake without hitting ElevenLabs.

4. **A session-attach protocol, with a fake session double that scripts a *sequence* of reply chunks, not one reply** — the single largest gap this chapter adds to Chapter 2. **Correction, from peer review:** an earlier draft specified the fake as returning one scripted reply value, which cannot exercise **FR-11** (speaking begins on the reply's first complete portion, not once the whole reply exists) — the exact behavior that makes a long agent turn (**UC-5**) feel alive. The fake must return a scripted sequence of reply chunks with independently controllable timing between them, mirroring what it stands in for: a stream, not a call-response pair. Structurally the same discipline as `FakeProgramGateway`

(records every injected transcript, caller controls what comes back) — but the "what comes back" shape has to be a stream to be useful. **This fake does not exist today and is new required work.**

5. **A playback-sink protocol, with an in-memory fake — necessary but not sufficient on its own. Correction, from peer review:** an earlier draft implied a call-recording fake alone makes the buffer-clear ordering questions (§2.5: stop-during-a-stop, buffer-clear racing a concurrent writer) unit-testable. It does not — those are genuinely concurrent races that only exist when two call sites contend, and a fake driven single-threadedly proves nothing about a race. The in-memory sink is still required (it removes the real-audio-output dependency), but it must be paired with a concurrency/interleaving harness — controlled `asyncio` task scheduling with deterministic yield points, or a threaded stress test run N times — that actually drives the race, not just a fake that records calls made from one thread.

6. **The call state machine is a pure class plus an injected policy, reachable only through a single serialized dispatch point. Correction, from peer review:** mirroring `voxd/programs/program.py`'s `Program` for the pure state/transition shape is right, but an earlier draft said to mirror it "exactly" without also copying the discipline that makes it safe: `Program` has no internal lock and does bare `self._state = self._state.with_updates(...)` assignments, safe only because `Program`'s own docstring states something upstream serializes every call into one writer. The call state machine's actual callers — turn detection, barge-in detection, and sentence-streamed synthesis — are three independently-running, continuous processes (§"Component overview") that can race exactly the way §2.5 already flags as unresolved. Copying `Program`'s pure-class shape without also requiring a single serialized dispatch point (a queue, an actor, or an explicit lock at the call boundary) reproduces a data race, not a design — this is a requirement on Chapter 2's orchestration layer, not something the pure class or its fake can supply alone.

7. **Presence-pool selection takes an injectable source of randomness / play-history**, not a bare call to `random.choice` at the call site. A test needs to assert anti-repeat behavior deterministically ("the same entry never plays twice in a row"), which requires controlling what "last played" and "next pick" resolve to, the same way any seeded-random test would.

8. **The async/sync bridge between `voxd`'s `async def` handlers and the synchronous STT/TTS provider calls must itself be injectable. Added per peer review:** an earlier draft of this chapter had no testability treatment at all for the boundary Chapter 2 names as "the dominant unsolved engineering problem" (§"Sentence-streamed synthesis"), despite applying exactly this discipline to the microphone (item 1) and the clock (item 2). Whatever bridges the boundary — a thread pool executor, subprocess isolation, or an async-native provider client — must be substitutable with (a) a fake that completes synchronously, proving the bridge introduces no deadlock, and (b) a fake that stalls, proving a slow synthesis call does not block the event loop for its duration.

3.3 Test coverage requirements

Corrected per peer review: items 1, 2, 3, and 5 below originally stated intent with no enforcement mechanism — compare to item 4, which was already enforceable the way `PL-TT-2` is elsewhere in this project's standards (a tool check, not a sentence). Each now names a concrete, greppable artifact.

1. **Every operation in the deferred Z specification has at least one Tier 1 test exercising it**, once that spec exists (Slice 1a). **Enforcement:** test names include the Z-spec operation name verbatim (per `docs/WORKFLOW.md`'s "assert the modeled properties by name"); a script diffs the operation names in the `.tex` Z spec against test names under `tests/conversation_mode/` and fails on any operation with zero matches. A state transition with no corresponding test is exactly the class of defect the Z-spec gate exists to prevent.

2. **Every functional and non-functional requirement in Chapter 1 maps to at least one test at a named tier. Enforcement:** a coverage-matrix file (FR/NFR id → tier → test path) checked in alongside the tests; a script diffs its FR/NFR ids against this document's **FR-** and **NFR-** labels and fails on any requirement missing a row. Where a requirement is genuinely only checkable by ear (NFR-6's "does this sound right," FR-12's presence-signal naturalness), the mapped tier is 5, not a missing row. **FR-19** is the counter-example worth naming explicitly: despite sounding like a judgment call, it is Tier-1-testable and cheap once the `STTProvider` fake (§3.2 item 3) exists — script a low-confidence result, assert the system asks the human to repeat rather than acting on it. Not everything audio-adjacent defaults to Tier 5.

3. **Every Known Unknown that gets resolved during implementation ships with a regression-style test or measurement harness, not just a one-time manual check. Enforcement:** Tier 3 regression harnesses live under a required, named directory (`tests/conversation_mode/harnesses/`) so their presence is greppable, not merely encouraged. Termination's buffer-tail latency (currently unmeasured) and **multi-session audio contention** (Chapter 2's Known Unknowns; untested by any spike, and the thing Spike 5 incorrectly suspected mid-session) are this item's two named worked examples — once either is resolved, its measurement becomes a harness in that directory, re-runnable, not a number recorded once and never checked again.

4. **Coverage never decreases on a touched file**, per this project's existing standard (`python-testing.md`, PL-TT-2: `pytest --co -q` before/after, count must not decrease) — restated here because Conversation Mode's components (detectors, the state machine, the segmenter) are exactly the kind of new pure logic where it is cheapest to hold this line from the first commit, not retrofit it later.

5. **The barge-in-vs-backchannel discrimination default (500 ms, FR-22) ships with its calibration methodology as a reusable Tier 3 harness**, not just the number. **Enforcement:** this harness is one of the named entries required under `tests/conversation_mode/harnesse` by item 3 above, not a separate, unenforced requirement. The number came from one operator's live-tuned preference in one room; the harness that produced it (play a real passage, stop on a sustained-duration threshold, ask for feedback, adjust) is what makes that default re-tunable for a different voice or room later, per §1.5's open question on this — ship the method, not just its output.